Encryption Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC6.1/CC6.7 · HIPAA §164.312(a)(2)(iv) + §164.312(e)(1)(A/B) · NIST SP 800-52r2 (TLS) · NIST SP 800-175B (algorithms) **Companion evidence:** HIPAA SRA v1 (`compliance/HIPAA_SRA_v1_2026-07-21.md`, railcall-core), Access Control Policy v1.0 ADOPTED (Probo).

1. Purpose

State the algorithms, key management, and boundary conditions under which RailCall encrypts data at rest, in transit, and in use. Written to match the reality of the product today — every algorithm cited is one the code actually uses, every key lifecycle described is one an operator can reproduce.

2. Scope

- Every path where RailCall touches customer credentials, receipts, entitlements, or PHI (which the product actively refuses to store — see §5).
- The signing seed that anchors receipt integrity and paid-tier entitlements — the SoT for downstream trust claims.
- The vault holding BYOK integration credentials.
- The gateway transport (`railcall-core.onrender.com`) and Postgres.

Encryption inside third-party providers we depend on (Render's managed Postgres, Stripe, Anthropic) is called out where it applies but is ultimately their responsibility surface — cross-referenced by SLA rather than reimplemented.

3. Algorithms in use

Purpose	Algorithm	Where
Receipt + entitlement signatures	Ed25519 (RFC 8032)	`railcall_signing.py`, `receipt_signer.py`, `entitlement.py`
Signature fast path	`cryptography` package native Ed25519	`railcall_signing.py:_sign_raw` — falls back to `ed25519_pure` when unavailable, byte-identical output per RFC determinism
Vault at rest (BYOK creds)	AES-256-GCM with a scrypt-derived key	`railcall-contrib/vault/vault.py:38-44,95-110`
Payload integrity hashing	SHA-256 (canonical JSON, sorted keys)	`receipt_signer.py:canonical_bytes`, `approval_airlock.py:40-45`
Audit chain linking	SHA-256 (`prev_hash` per record)	`workbench/audit_chain.py`
Blind seat auth over the wire	SHA-256(api_key) → hex	`cloud_gateway.py` `/v1/seat/checkin`, `/v1/meter`
Transport	TLS 1.3 to `railcall-core.onrender.com` (Render-terminated)	`RAILCALL_GATEWAY_URL` set to https:// in the CLI; loopback traffic on the station never leaves the host
OS keystore-backed seed protection	macOS Keychain (Data Protection API) / Windows DPAPI / Linux Secret Service	`seed_store.py`

Algorithms deliberately NOT used (worth stating so a review can't misread the absence):

- **RSA:** not used for RailCall-generated keys. The only RSA presence is Probo's OAuth2 signing key in the dev infra tree, out of RailCall scope.
- **SHA-1 / MD5:** not used anywhere in RailCall code paths.
- **Symmetric key exchange in the field:** not used — we do not run a session-key protocol; every symmetric encryption is a local at-rest concern.

4. Key management

4.1 The install signing seed (paid-tier root of receipt trust)

- **Generation:** 32 bytes from `os.urandom` (CSPRNG) at first station boot.
- **Storage:** OS keychain via `seed_store.py`. macOS `security(1)` login keychain (secret piped over stdin, never argv). Windows DPAPI encrypts the seed with the current user's credentials and writes the ciphertext beside the vault. Linux libsecret via `secret-tool`.
- **Fallback:** if no keystore backend is available (e.g., Linux headless without libsecret), the seed stays in the 0600 vault file. `seed_store.status()` reports `at_rest="plaintext_file"` — the honest posture, never a false claim of protection.
- **Migration:** on every boot, `seed_store.migrate()` idempotently moves any pre-existing plaintext seed into the keystore. **Ordering is a safety property:** the keystore write must succeed BEFORE the plaintext copy is dropped, or a failed migration would orphan the install identity. `test_seed_store.py:5b` asserts this.
- **Rotation:** via `railcall rotate-key`. Publishes the new pubkey, archives the previous public doc for verifier continuity, writes the new seed through `seed_store` in the same place `_ensure_signing_seed` reads. Rotation is user-initiated only; there is no automatic rotation.
- **Never leaves the box:** the private seed is never displayed, logged, or transmitted. `install.sh` explicitly excludes `keys.local.json` from the file set it distributes.

4.2 The issuer seed (paid-tier authority)

- **Generation:** via `tools/mint_issuer_keypair.py` — one-shot ceremony, refuses to silently discard the seed.
- **Storage:** environment variable `RAILCALL_ISSUER_SEED` on the Render gateway service. Set service-level, NOT via env groups (an env group must be explicitly linked to a service before values flow; direct service-level env vars trigger a redeploy on save).
- **Backup:** 1Password + offline copy. Losing the seed is catastrophic (see §7).
- **Rotation:** any rotation invalidates every outstanding entitlement. Documented as a deliberate, human-only operation — no automation.
- **Reachability:** `/v1/issuer/pubkey` (public) returns the derived pubkey; the seed itself never appears in any API response, never in any log line, never in any error message. Missing seed → endpoints fail closed with 503 (`cloud_gateway.py:_issuer_seed`).

4.3 The BYOK vault (integration credentials)

- **Encryption at rest:** AES-256-GCM + scrypt KDF, opt-in per integration.

- **File permissions:** 0600 in all cases, unencrypted or encrypted; pinned at fd creation (`vault_io.py` — `os.fchmod` before any secret bytes land).
- **Atomicity:** every write is temp → fsync → `os.replace` → chmod → dir fsync. A crash never leaves a truncated vault; a partially-written file cannot masquerade as an empty vault (`vault_io.load()` raises `VaultCorruptError` rather than silently returning `{}`).
- **Reserved keys:** `_railcall_signing_seed` is a reserved vault key filtered out of every integration listing (`railcall_signing.is_reserved_vault_key`).

4.4 Session tokens (studio access)

- Per-process CSRF token (32 bytes hex), embedded in the served page. Lives for the process lifetime, discarded on studio exit.
- Loopback-only bind (`127.0.0.1`) is the primary access boundary; the token is a secondary defence against cross-origin fetches.
- The "approve code" for irreversible actions is printed to the launching terminal only — never templated into any served page — so a compromised browser cannot self-approve. See Access Control Policy §3.

4.5 Stripe API keys

- Live keys held in Render env vars, `sync:false` (per-environment, never in the repo).
- Restricted keys preferred over standard keys for any scoped operation (Products+Prices write for price setup, etc.). Restricted keys are treated as single-use and revoked after the task that needs them.

---

5. What we do NOT encrypt, and why

- **Customer workflow inputs and outputs:** the product does not store these. They live on the customer's own machine in files the customer controls. Encryption of that data is the customer's concern, not ours. This is not a gap — it is architectural. Stated in the SRA §1 ("Key architectural fact"), stated in the BAA Exhibit A (data flows).
- **PHI in receipt bodies:** does not appear, by construction. `phi_guard.py` + `approval_airlock.py:redact()` remove HIPAA identifiers before the receipt is signed — three-pass (credentials by field name → identifiers by field name → identifiers by value), with auto-escalation on clinical field-name signals. `test_phi_guard.py` 29/29. This is a stronger property than encryption at rest — the sensitive data never reaches the write path in the first place.
- **PHI in gateway requests:** does not appear, by contract. `/v1/seat/checkin` and `/v1/meter` accept a fixed set of scalar fields (blind key hash + install pubkey + nonce, or key hash + run count + nonce); `/v1/attestation/countersign` accepts an integrity hash and an attestation

id. No endpoint accepts a receipt body or payload. Contract-violation risk documented in SRA §5.3.

- **Free-tier metered columns in the gateway DB:** `consumers.free_runs_remaining` / `runs_used` are integers, not sensitive. Not encrypted at the column level; protected by DB-level access via Render.

---

6. Transport

- **CLI → gateway:** HTTPS to `https://railcall-core.onrender.com`, TLS 1.3 negotiated with Render's certificates. No HTTP fallback.
- **Station → gateway:** same.
- **Browser → studio:** loopback only (`http://127.0.0.1:...`), no TLS. Justified by the loopback bind — the traffic physically cannot leave the host, so TLS would add complexity without adding protection.
- **Station internal (workflow engine → integrations):** governed by the customer's own configuration. RailCall's role is redaction before egress (§5), not termination-point encryption.

---

7. Loss / compromise procedure

Written per-key so an operator under stress has a script to follow, not prose to parse.

Install signing seed lost (single install): 1. That install can no longer sign new receipts. Existing receipts remain verifiable because the pubkey is published (`signing_pubkey.json`). 2. Run `railcall rotate-key` — mints a fresh keypair, archives the old public doc to `signing_pubkey.prev-<ts>.json`, publishes the new one. 3. Pre-rotation receipts still verify via `verify --key <archived>`.

Install signing seed suspected COMPROMISED: 1. Same as loss, PLUS: notify anyone who trusts receipts from that install that receipts dated between compromise-suspected and rotation should be treated as unverified. 2. If unclear whether receipts were forged post-compromise, treat the whole install as compromised — wipe, reinstall, generate a fresh identity.

Issuer seed lost: 1. Every outstanding entitlement becomes unreplaceable. Existing installs continue to work with the entitlement they already hold until it expires. 2. Recovery: `tools/mint_issuer_keypair.py mint --seed-out <path>`, back up the new seed, edit `ISSUER_PUBKEY_HEX` in `railcall-engine/workbench/primitives/entitlement.py`, cut a new station release (see `RELEASE_station-v0.16.md` for the ceremony), re-pin `install.sh` + `install.ps1`, deploy the mirror, set the new seed as `RAILCALL_ISSUER_SEED` on Render. 3. Every existing paying customer must re-run the installer to get a station that trusts the new key. Communicate this via email + release notes before deploying.

Issuer seed suspected COMPROMISED: 1. Same as loss, PLUS: rotate immediately (do not wait to build the new station release — a compromised seed lets an attacker mint valid-looking entitlements today). 2. Every entitlement minted since compromise-suspected is untrusted. Communicate the rotation to affected customers and re-mint their entitlements against the new authority.

BYOK vault entry compromised: 1. Customer's concern (the credentials are theirs). Actionable steps: rotate the compromised credential in the upstream service, remove the entry from `keys.local.json`, re-authorize. 2. RailCall the operator has no visibility into which BYOK credentials exist — nothing to notify.

8. Third-party encryption we rely on

Provider	What we entrust	Their responsibility
Render	Gateway Postgres, service logs, TLS termination	Encryption at rest (managed Postgres), TLS 1.3 in transit, secret storage for `sync:false` env vars
Stripe	Payment card numbers, subscription state	PCI-DSS Level 1; RailCall never sees full card numbers
Anthropic (Probo LLM only)	Prompt text sent from Probo's internal agents	Anthropic API contract — no customer PHI reaches this path because Probo is an internal tool, not customer-facing
GitHub	Source code, release artifacts, station tarball	Standard GitHub SLA; no secrets in the repo (verified by the release script's leak gate)
VPS 157.230.177.45	railcall.ai static site + installer mirror	SSH-key-authenticated access only; no customer data on the box

9. Review

Reviewed at each station release AND at least annually. Algorithm choices are re-reviewed against NIST SP 800-131A (retired algorithms) at each annual review. Any change to §3 (algorithms) requires a corresponding change here BEFORE the code lands.

10. Related controls

- Signing seed at rest: measure `P0-1` (`seed_store` integration)
- Audit trail integrity: measure `P0-2` (`audit_chain`), enforces §164.312(b)
- PHI egress prevention: measure `P0-3` (`phi_guard`), enforces §164.312(e)(1)

- Person authentication (agent-side): Access Control Policy §4.2
- Loss-of-trust procedures: this policy §7